

[問題19] (抽象データ型の設計における留意事項)

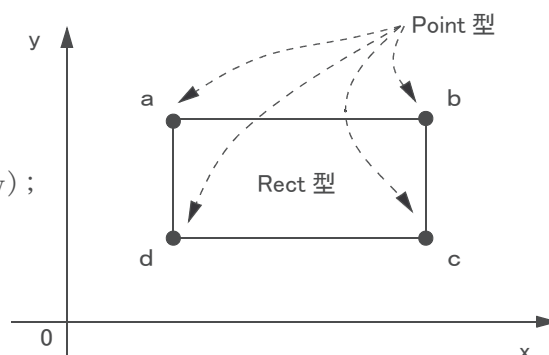
右下図のような2次元平面上の長方形を取り扱うプログラムを、データ抽象化の考え方に基づいて作りたいと思います。

まず、長方形というデータを表現するためには、長方形そのものを表現するデータ型と、それを構成する点を表現するためのデータ型が必要になります。そこで、点と長方形を表現する抽象データ型として、それぞれ **Point** 型と **Rect** 型を定義することにします。

Point 型と **Rect** 型を扱う操作関数には、以下のものを用意することにしました。以下の各問いに答えなさい。

(Point型の操作関数)

- ① `double ptGetX(Point p);`
機能：点pの x 座標を返す
- ② `double ptGetY(Point p);`
機能：点pの y 座標を返す
- ③ `Point ptCreate(double x, double y);`
機能：座標 (x, y) の点を生成する



(Rect型の操作関数)

- ④ `double rctArea(Rect rct);`
機能：長方形rctの面積を返す
- ⑤ `double rctPerim(Rect rct);`
機能：長方形rctの周囲の長さを返す
- ⑥ `double rctWidth(Rect rct);`
機能：長方形rctの幅(横)の長さを返す
- ⑦ `double rctHeight(Rect rct);`
機能：長方形rctの高さ(縦)の長さを返す
- ⑧ `Point rctTopLeftCorner(Rect rct);`
機能：長方形rctの左上隅の点を返す
- ⑨ `Rect rctCreate1(Point p1, Point p2);`
機能：対角線を表す2点p1とp2から、長方形を生成する
- ⑩ `Rect rctCreate2(Point p, double width, double height);`
機能：左上の1点pと、幅(横)widthおよび高さ(縦)heightの長さから、長方形を生成する

- (1) **Rect** 型の操作関数⑨と⑩は、どちらか一方があれば④～⑧の操作関数を実装することができます。それにも関わらず、このような冗長な関数を用意する理由は何ですか。
- (2) **Point** 型の操作関数①～③と **Rect** 型の操作関数④～⑩のうち、各データ型のデータ構造を決定しなければ実装することができないものがあります。どれですか？(このように、データ構造を直接参照する操作関数を、アクセサー: **accessor** といいます)。

- (3) **Point** 型と **Rect** 型のデータ構造を次のように定義しました。**Point** 型と **Rect** 型の操作関数をすべて実装しなさい。

但し、操作関数のうち(2)で挙げたもの以外は、これらのデータ構造を直接参照しないようにすること。

(Point型のデータ構造)

```
typedef struct {
    double x; // x 座標
    double y; // y 座標
} Point;
```

(Rect型のデータ構造)

```
typedef struct {
    Point p1; // 対角線の端点1
    Point p2; // 対角線の端点2
} Rect;
```

- (4) 長方形の対角線の長さを求める操作関数 **rctDiaglen()**を作りなさい。ただし、外部仕様は次のようにします。

(Rect型の操作関数: 追加)

- ① `double rctDiaglen(Rect rct);`
機能: 長方形 `rct` の対角線の長さを返す

- (5) 適当な2つの長方形を入力し、その面積と周囲の長さ、および対角線の長さを出力するプログラムを、適切な操作関数を利用して作りなさい。

但し、長方形のうち一つは対角線の2点を、もう一つは長方形の左上の点および縦と横の長さを指定して初期化すること。

(prog 19-05. c)

(実行例)

対角線の端点1の座標 (x, y) : 10, 10 

対角線の端点2の座標 (x, y) : 30, 50 

左上隅の座標 (x, y) : (10, 50), 面積 : 800, 周囲 : 120, 対角線長 : 44.7214

左上の点の座標 (x, y) : 10, 10 

横 : 40 

縦 : 30 

左上隅の座標 (x, y) : (10, 10), 面積 : 1200, 周囲 : 140, 対角線長 : 50

- (6) **Rect** 型のデータ構造を以下のように変更しました。(5) のプログラムを作り直しなさい。但し、操作関数の外部仕様には変更を加えないものとします。

(prog 19-06. c)

(変更したRect型のデータ構造)

```
typedef struct {
    Point p;          // 長方形の左上の点
    double width;     // 長方形の幅 (横の長さ)
    double height;    // 長方形の高さ (縦の長さ)
} Rect;
```

(7) (6)のプログラムで、修正が必要になった操作関数をすべて列挙しなさい。

注意: ここで、もし(4)で作った `rectDiaglen()` 関数が含まれていたら、`rectDiaglen()` 関数の作り方が適切でなかったことを意味します。その場合は、(6)の変更の影響を受けないように `rectDiaglen()` 関数を作り直しなさい。

(8) `Point`型のデータ構造を以下のように変更しました。(5)のプログラムを作り直しなさい。但し、操作関数の外部仕様には変更を加えないものとします。

(prog 19-08. c)

(変更した`Point`型のデータ構造)

```
typedef struct {  
    double p[2]; // p[0]: x座標, p[1]: y座標  
} Point;
```

(9) (8)のプログラムで、修正が必要になった操作関数はどれですか？

注意: ここで、もし、修正が必要な操作関数の中に`Rect`型の操作関数が含まれていたら、それは(2)の解答が間違っているか、または(3)で作った`Rect`型の操作関数の作り方が適切でなかったことを意味します。その場合は、(8)の変更の影響を受けないように作り直しなさい。

(10) (1)～(9)から、抽象データ型を設計する(データ抽象化の考え方に基づいてプログラミングを行う)際に、どのようなことに注意すべきですか。データの外部仕様の設計と内部仕様の実装、それぞれについてまとめなさい。

レポート

次のものをE-mailで提出しなさい。

- ・[問題19](5)(6)(8)のプログラム(添付ファイル)と実行結果(本文)
- ・[問題19](1)(2)(7)(9)(10)の解答(本文)
- ・演習の感想(本文)

(提出先) E-mail: k-kenta@hamako-ths.ed.jp 件名: プロ技演習19

(提出期限) / ()

(参考) 問題(5)のプログラムの先頭部分は、例えば次のようになります。ここで、独自に作った構造体や型は、それが使われる前に定義されている必要があることに注意してください。特に、**Point** 型は **Rect** 型を定義するのに使われるため、**Rect** 型よりも先に定義しておく必要があります。

なお、このプログラムの **main()**関数部分までは `/home/pub/sample/prog19.c` にありますので、適宜コピーして利用して下さい。

○ 問題(5)のプログラムの先頭部分

```
#include <stdio.h>
#include <math.h>

typedef struct {
    double x; // x 座標
    double y; // y 座標
} Point;
```

```
typedef struct {
    Point p1; // 対角線を表す点1
    Point p2; // 対角線を表す点2
} Rect;
```

```
Point ptCreate(double x, double y);
double ptGetX(Point p);
double ptGetY(Point p);

double rctArea(Rect rct);
double rctPerim(Rect rct);
double rctWidth(Rect rct);
double rctHeight(Rect rct);
double rctDiaglen(Rect rct);
Point rctTopLeftCorner(Rect rct);
Rect rctCreatel(Point p1, Point p2);
Rect rctCreate2(Point p, double width, double height);
```

```
main()
{
    Rect r;
    Point p1, p2;
    double x1, y1, x2, y2, w, h, s;

    printf("長方形 1\n");
    printf("対角線の端点1の座標(x, y) : "); scanf("%lg, %lg", &x1, &y1);
    printf("対角線の端点2の座標(x, y) : "); scanf("%lg, %lg", &x2, &y2);
    p1 = ptCreate(x1, y1);
    p2 = ptCreate(x2, y2);

    r = rctCreatel(p1, p2); // 長方形の対角線を表す2点で初期化

    ... 以下略 ...
}
```

(数学関数の利用について)

必要に応じて以下の数学関数を利用しても構いません。

<code>pow(x, y)</code>	: x の y 乗	x^y
<code>fabs(x)</code>	: x の絶対値	$ x $
<code>sqrt(x)</code>	: x の平方根	$\sqrt{x}$